Models for Irregular Time-Series in Healthcare

Yilin Li

Contents

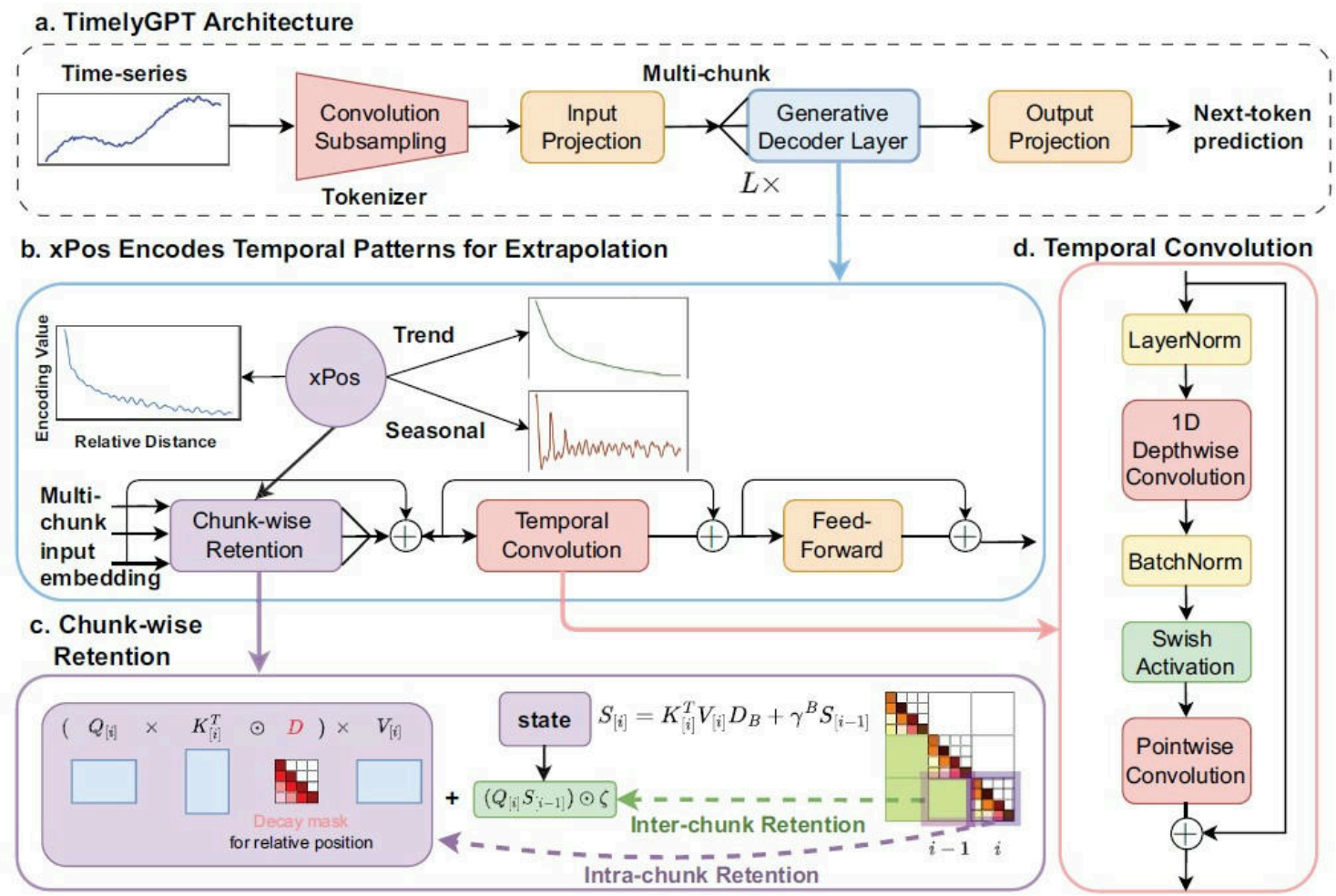
1. TimelyGPT
2. TrajGPT
3. Comparison and Conclusion

TimelyGPT

TimelyGPT: Overview

- **Paper**: TimelyGPT: Extrapolatable Transformer Pre-training for Long-term Time-Series Forecasting in Healthcare[1]
- **Goal**: To effectively pre-train on large-scale time-series data, including both continuously monitored biosignals and irregularly-sampled clinical records, for long-term forecasting tasks.
- **Core Idea**: We will explore three main components:
  1. **xPos Positional Encoding**: For capturing trend and seasonal patterns.
  2. **Retention Mechanism**: For efficient global dependency modeling.
  3. **Convolution Modules**: For capturing local interactions.

TimelyGPT: Overview



TimelyGPT: xPos Positional Encoding

- Given an input embedding $X \in \mathbb{R}^{N \times d}$ for N tokens at d embedding dimensions, xPos is integrated into the n-th token embedding X_n through rotation matrix $e^{i\theta n}$ and exponential decay γ^n:

$$\tilde{Q}_n \tilde{K}_m = X_n W_Q \left(\gamma e^{i\theta}\right)^{n-m} X_m W_K = \gamma^{n-m} \hat{Q}_n \hat{K}_m$$

where $\hat{Q}_n = X_n W_Q e^{i\theta n}$, $\hat{K}_m = X_m W_K e^{-i\theta m}$, θ and γ indicate position-dependent rotation and decay hyperparameters.

(2)

- The exponential decay γ^{n-m} determines the intensity of remembering historical information, while the rotation matrix $e^{i\theta n}$ captures the oscillation frequencies.
- This decay mechanism effectively enables the model to process sequences longer than the training set.

TimelyGPT: Retention Mechanism

- The **Retention** mechanism is an efficient attention and memory module that can be formulated as an RNN.
- To accommodate irregularly-sampled time series, the decay mask D is adapted according to the time gap $\Delta t_{n,m} = t_n - t_m$. The parallel form is:

$$\mathrm{Ret}(X) = (QK^T \odot D)V, \quad D_{nm} = \begin{cases} \gamma^{\Delta t_{n,m}} & t_n \geq t_m \\ 0 & t_n < t_m \end{cases} \tag{4}$$

- For next-token pre-training, this is expressed using a recurrent formula that incorporates the time gap $\Delta t_{n,n-1}$ into the state variable S_n:

TimelyGPT: Retention Mechanism

$$S_n = \gamma^{\Delta t_{n,n-1}} S_{n-1} + K_n^T V_n$$

$$\mathrm{Ret}(X_n) = Q_n S_n \tag{5}$$

- Finally, retention will be added to the input embedding in the form of residual learning

TimelyGPT: Convolution Modules

- TimelyGPT uses convolution modules to effectively capture nuanced **local interactions** in the time series data.
- The paper proposes a temporal convolution module built upon **depthwise separable convolution**, a computationally efficient alternative to standard convolution. As shown in figure, this process is split into two main layers:

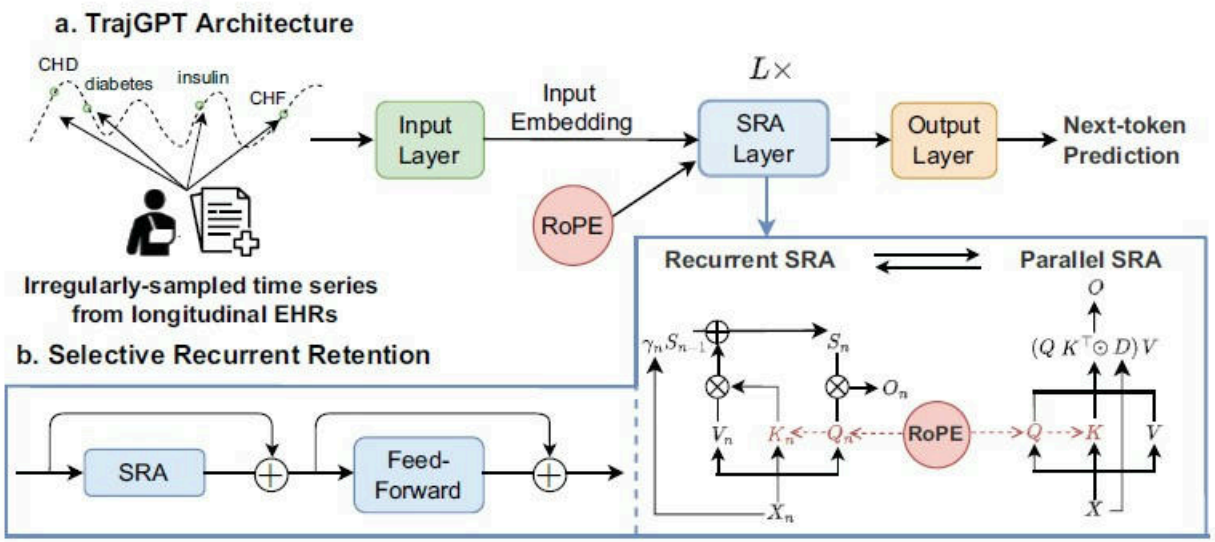
1. **1D Depthwise Convolution**:
   - This layer applies a single convolution filter to each input channel (feature map) **independently**. If there are d input channels, it uses d separate filters, one for each channel.
2. **Pointwise Convolution (1x1 Convolution)**:
   - This is a standard convolution but with a kernel size of 1. It takes the output from the depthwise layer and operates across all channels at each time step.

TRAJGPT

TrajGPT: The Core Architecture

- Paper: TrajGPT: Irregular Time-Series Representation Learning for Health Trajectory Analysis[2]
- **The Main Innovation**: TrajGPT's key upgrade is replacing the **data-independent** decay of TimelyGPT with a **data-dependent**, selective decay mechanism. This is achieved through a new module called Selective Recurrent Attention (SRA).

TrajGPT: The Core Architecture



SRA: Data-Dependent Decay Mechanism

- First, a Rotary Position Embedding (RoPE) is added to the token embedding. It handles varying time intervals by encoding the relative distance $t_n - t_m$. The Query, Key, and Value matrices are then computed as:

$$Q_n = X_n W_Q e^{i\theta t_n}, \quad K_m = X_m W_K e^{-i\theta t_m}, \quad V_m = X_m W_V$$

- In the recurrent forward pass, the SRA module uses these matrices to compute the output representation O_n based on a state variable S_n:

$$S_n = \gamma_n S_{n-1} + K_n^T V_n, \quad O_n = Q_n S_n$$

where $\gamma_n = \mathrm{Sigmoid}\left(X_n w_\gamma^T\right)^{\frac{1}{\tau}}$

SRA: Data-Dependent Decay Mechanism

- The key is the **data-dependent decay** $\gamma_n \in (0, 1]$, which is calculated from the input X_n and a learnable vector w_γ.
- This enables SRA to selectively forget irrelevant past information based on contexts. For chronic diseases, TrajGPT learns higher γ_n values to slow down forgetting. For acute conditions, it learns lower γ_n values to prioritize recent events.
- Given an initial state $S_0 = 0$, the recurrent update of S_n can be rewritten as a sum of contributions from all previous timesteps:

$$S_n = \sum_{m=1}^{n} \left(\prod_{t=m+1}^{n} \gamma_t \right) K_m^T V_m = \sum_{m=1}^{n} \left(\frac{b_n}{b_m} \right) K_m^T V_m, \quad \text{where} \quad b_n = \prod_{t=1}^{n} \gamma_t$$

- To express this in matrix form, a causal decay matrix D is introduced:

SRA: Data-Dependent Decay Mechanism

$$D = \begin{pmatrix} \frac{b_1}{b_1} & 0 & \dots & 0 \\ \frac{b_2}{b_1} & \frac{b_2}{b_2} & \dots & 0 \\ \dots & \dots & \dots & \dots \\ \frac{b_N}{b_1} & \dots & \dots & \frac{b_N}{b_N} \end{pmatrix} = \begin{pmatrix} 1 & 0 & \dots & 0 \\ \gamma_2 & 1 & \dots & 0 \\ \dots & \dots & \dots & \dots \\ \prod_{t=2}^{N} \gamma_t & \dots & \dots & 1 \end{pmatrix}$$

- With the summation form and the decay matrix D, we reformulate the output O_n into matrix form:

$$O_n = Q_n \sum_{m=1}^{n} \left(\frac{b_n}{b_m} \right) K_m^T V_m = ((Q_n K^T) \odot D_n) V$$

SRA: Data-Dependent Decay Mechanism

- By computing output representations for all tokens, the **parallel form** of SRA is given by:

$$O = (QK^T \odot D)V$$

SRA as a Discretized Neural ODE

- The derivation begins with the definition of a **continuous-time State Space Model (SSM)**, which is a first-order ODE:

$$S'(t) = AS(t) + BX(t), \quad O(t) = CS(t)$$

- Using a zero-order hold (ZOH) discretization rule [3], this continuous model is converted into a **discrete-time model**:

$$S_t = \bar{A}S_{t-1} + \bar{B}X_t, \quad O_t = CS_t$$

where $\bar{A} = e^{\Delta A}$ and $\bar{B} = \left(e^{\Delta A} - I\right)A^{-1}B$.

SRA as a Discretized Neural ODE

- The key insight is that the recurrent form of SRA is a special case of this discrete-time SSM. The SRA update is written as:

$$S_t = \Lambda_t S_{t-1} + K_t^T V_t, \quad O_t = Q_t S_t$$

- This directly corresponds to the discrete SSM with the parameters:

$$\bar{A}, \bar{B}, C) = (\Lambda_t, K_t^T, Q_t)$$

- Based on this correspondence, the parameters for the underlying continuous-time SSM can be derived:

SRA as a Discretized Neural ODE

$$C = Q_t, \quad A = \frac{\ln(\Lambda_t)}{\Delta}, \quad B = A\left(e^{\Delta A} - I\right)^{-1} K_t^T$$

- Since the ODE parameters (A, B, C) depend on the input observation X_t, the SRA module is effectively a discretized version of a **Neural ODE**[4]. The continuous dynamics are modeled as:

$$S'(t) = AS(t) + BX(t) = f(S(t), t, \theta_t)$$

- Therefore, a multi-head TrajGPT can be seen as a collection of discretized ODEs, with each head capturing distinct dynamics.

Comparison and Conclusion

Conclusion: TimelyGPT vs. TrajGPT

TimelyGPT: A Foundation for Irregular Time Series

- **Architecture**: A decoder-only Transformer model designed for generative pre-training.
- **Positional Encoding**: Uses **xPos**, which combines RoPE's **rotation** with a **data-independent exponential decay** ($\gamma^{\Delta t}$) to model both seasonal and trend patterns.
- **Core Mechanism**: Employs the **Retention** mechanism, an efficient RNN-like

TrajGPT: An Advanced Successor

- **Architecture**: Also a decoder-only model
- **Positional Encoding**: Simplifies the position encoding back to a standard **RoPE** (rotation only).
- **Core Mechanism**: Replaces Retention with **Selective Recurrent Attention (SRA)**. The key innovation is moving the decay mechanism into the SRA module and making it **data-dependent** (γ_n), allowing the model to se-

Conclusion: TimelyGPT vs. TrajGPT

attention module, to handle long sequences.

- **Local Patterns**: Integrates **Convolution Modules** to capture local interactions and features within the time series.

lectively forget information based on context.

- **Theoretical Shift**: Explicitly interpreted as a discretized **Neural Ordinary Differential Equation (ODE)**, providing a stronger theoretical basis for modeling continuous dynamics. It notably **removes the convolution modules**, relying entirely on the SRA's capabilities.

Summary and Outlook

TrajGPT can indeed be viewed as an advanced, “Plus” version of TimelyGPT. It refines the core ideas by making the crucial mechanism of temporal decay adaptive and grounding the entire architecture in the robust mathematical framework of Neural ODEs.

Both models are exceptionally well-suited for modeling long-term time series with high-frequency or irregular sampling intervals. As you noted, their demonstrated ability to capture complex, non-linear dynamics makes them highly promising candidates for application in challenging domains like quantitative finance and high-frequency trading analysis.

Bibliography

[1] Z. Song, Q. Lu, H. Xu, H. Zhu, D. L. Buckeridge, and Y. Li, "TimelyGPT: Extrapolatable Transformer Pre-training for Long-term Time-Series Forecasting in Healthcare." [Online]. Available: https://arxiv.org/abs/2312.00817

[2] Z. Song, Q. Lu, H. Zhu, D. Buckeridge, and Y. Li, "TrajGPT: Irregular Time-Series Representation Learning for Health Trajectory Analysis." [Online]. Available: https://arxiv.org/abs/2410.02133

[3] A. Gu, K. Goel, and C. Ré, "Efficiently Modeling Long Sequences with Structured State Spaces." [Online]. Available: https://arxiv.org/abs/2111.00396

[4] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud, "Neural Ordinary Differential Equations." [Online]. Available: https://arxiv.org/abs/1806.07366

scNODE

scNODE (single-cell Neural Ordinary Differential Equation) is an end-to-end deep learning model whose core objective is to learn the continuous temporal dynamics of cellular states from single-cell transcriptomic data observed at discrete time points, thereby enabling the prediction of gene expression profiles at any unobserved time point.

Problem: Cell Trajectory Inference

How can we utilize single-cell gene expression data, collected at a few discrete time points, to generate (predict) the real and complete gene expression profiles of a cell population at any unobserved time point of interest?

Objective: The primary task of scNODE is to serve as a generative model to predict single-cell gene expression data at unobserved time points in silico.

Two Tasks: This includes two scenarios:

Interpolation: Predicting “missing” time points that lie between two observed time points. For example, if we have data from Day 2 and Day 4, the model needs to predict the data for Day 3.

Extrapolation: Predicting future time points that extend beyond the observed time range. For example, if we have data up to Day 4, the model needs to predict the data for Day 5 or Day 6.

Problem: Cell Trajectory Inference

Background and Challenges: The data is sparse and discontinuous.

Cell Non-correspondence: A key challenge is that scRNA-seq experiments are destructive; each measurement lyses the cells. This means that the cells observed at different time points are entirely different batches of cells. Therefore, the model cannot assume it can track the trajectories of individual cells but must instead learn the evolutionary dynamics of the entire cell population.

Problem: Cell Trajectory Inference

Input and Output of the Problem:

Input: The model receives a time series of gene expression matrices.Output

Predict: For any given time point (t), the model must be able to generate a new gene expression matrix. This matrix should contain a large number of cells and be statistically similar to the true distribution of the cell population.

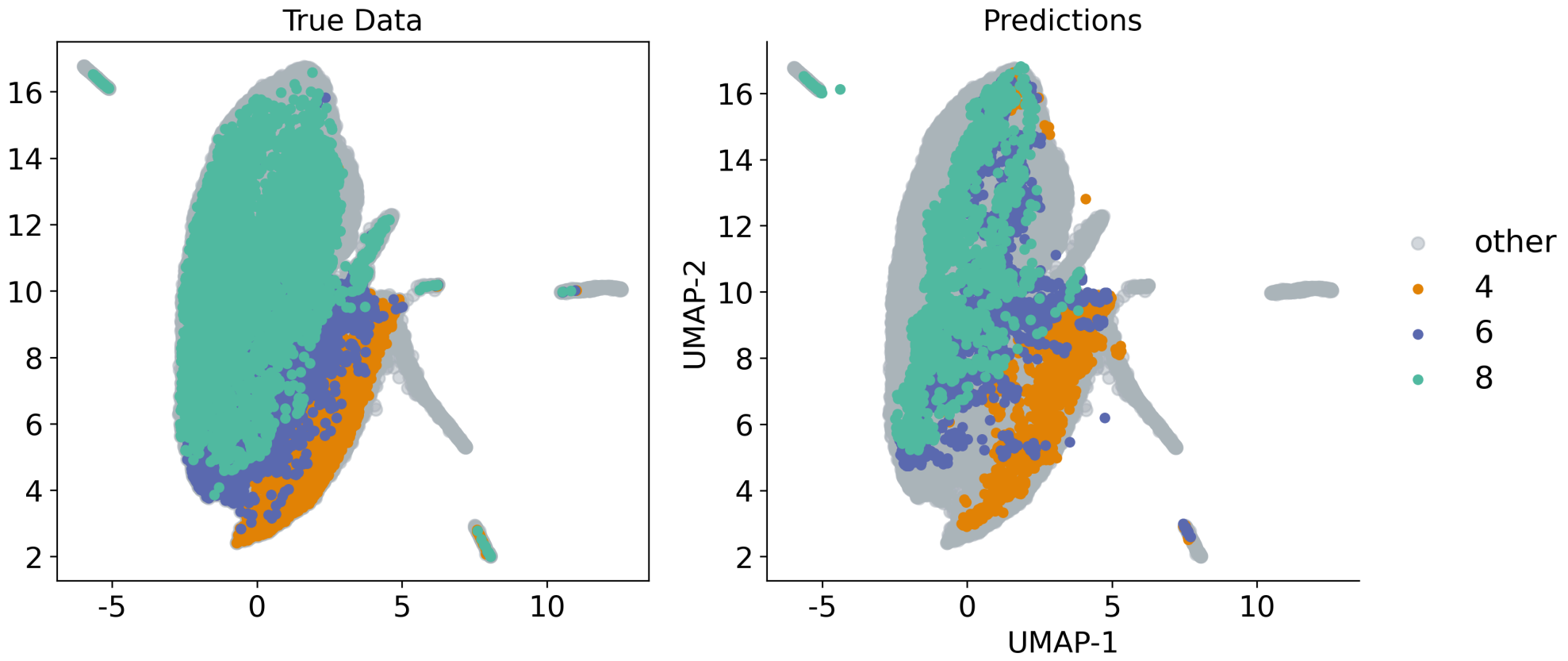
Algorithm

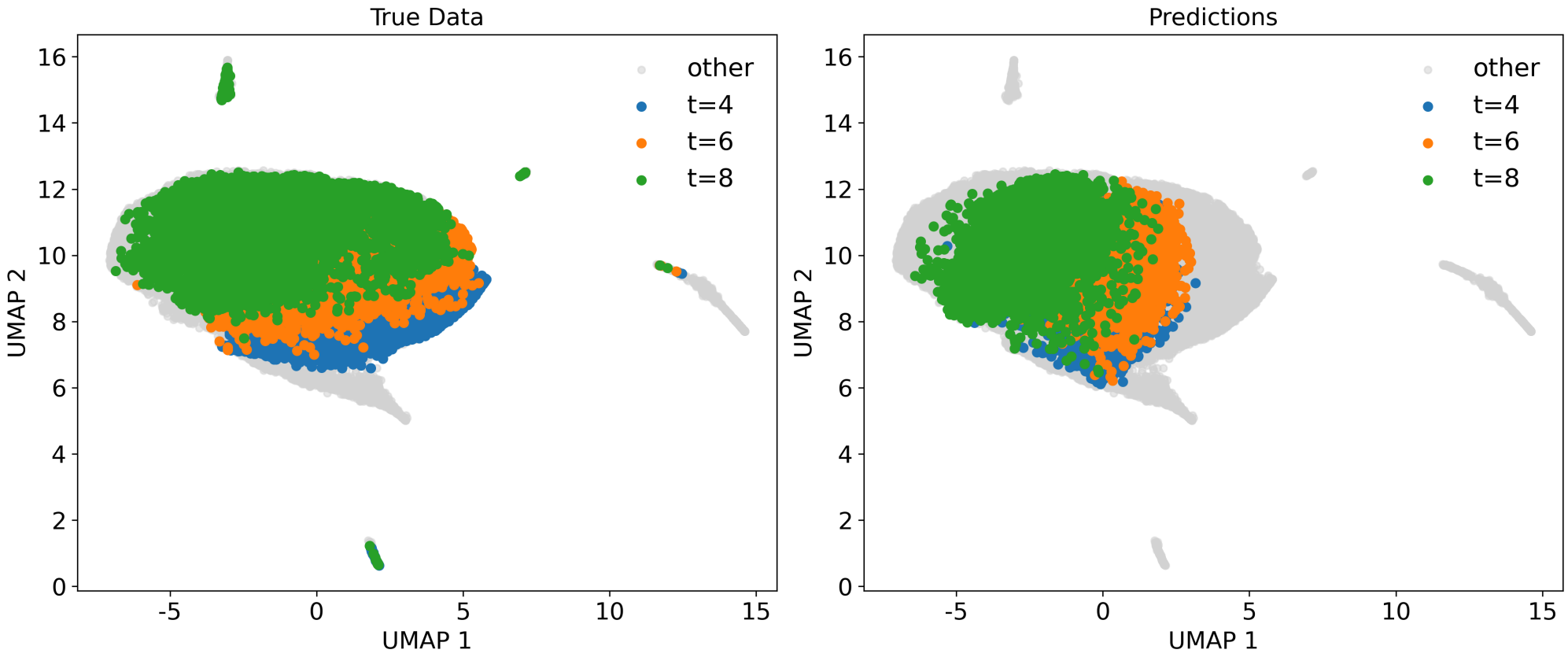
For scNODE, the model generates a virtual cell population at initial time point t_0 using a VAE, then it continuously extrapolate the cell state with a Neurl ODE

For TrajGPT, we first apply a t-SVD decomposition on whole dataset, then train the TrajGPT on this time series, at last the inverse t-SVD transformation is applied on predicted data.

Applying TrajGPT to Single-Cell Data: Results

The figure below shows the results generated by scNODE:



Applying TrajGPT to Single-Cell Data: Results

The figure below shows the results generated by TrajGPT: